

# Join, Subquery & Normalisasi Data, Window Function

PTP Program





# Training Guidelines

Untuk mengikuti materi pelatihan berikut ada beberapa hal yang harus diperhatikan oleh peserta:

- Peserta diharapkan sudah memiliki pengetahuan dasar tentang **database** dan **SQL**.
- Peserta diharapkan memperhatikan materi dan mencatat materi atau penjelasan yang disampaikan dari instruktur.
- Instruktur diharapkan dapat menjelaskan materi dengan jelas dan memastikan peserta pelatihan dapat memahami topik yang dibahas.
- Untuk contoh latihan dan kode atau syntax serta beberapa penjelasannya, anda dapat melihat di **Exercises**



## Point of Discussion

- Memahami Normalisasi dan Denormalisasi Data
- Memahami Cara Kerja dan Cara Menggunakan JOIN
- Memahami Cara Menerapkan Subquery
- Memahami Penggunaan Window Function



# Normalisasi dan Denormalisasi Data

## Normalisasi Data

# Apa itu Normalisasi Data?

**Normalisasi** adalah proses pengorganisasian data dalam basis data untuk mengurangi **redundansi** (pengulangan data) dan meningkatkan **integritas**. Proses ini melibatkan pemecahan tabel menjadi bentuk yang lebih kecil dan memastikan hubungan antar tabel terstruktur dengan baik.

Normalisasi dilakukan melalui beberapa tahap yang disebut **bentuk normal** (Normal Forms). Setiap tahap memiliki aturan tertentu untuk memenuhi kriteria normalisasi.

Berikut adalah tahapan umum yang biasanya digunakan untuk normalisasi data:

1. **1NF (First Normal Form)**: Hilangkan data duplikasi dalam satu kolom.
2. **2NF (Second Normal Form)**: Hilangkan ketergantungan sebagian pada primary key.
3. **3NF (Third Normal Form)**: Hilangkan ketergantungan transitif antar kolom.

## Normalisasi Data

# Kegunaan Normalisasi

Berikut ini adalah beberapa manfaat normalisasi data:

- **Menghindari Anomali Data:**

- **Anomali Insert:** Data tidak bisa dimasukkan karena ketergantungan pada data lain.
- **Anomali Update:** Perubahan pada satu data memerlukan banyak pembaruan di tempat lain.
- **Anomali Delete:** Penghapusan data tertentu menyebabkan kehilangan data yang seharusnya tetap ada.

- **Meningkatkan Integritas Relasional:**

Relasi antar tabel lebih terstruktur, sehingga mempermudah pengelolaan hubungan antar data.

- **Efisiensi Penyimpanan:**

Mengurangi redundansi, sehingga penggunaan ruang penyimpanan lebih hemat.

- **Kemudahan Querying:**

Struktur tabel yang modular mempermudah pembuatan query yang kompleks.

## Normalisasi Data

# Contoh Normalisasi

Bayangkan kita sedang membangun aplikasi manajemen restoran. Aplikasi tersebut perlu menyimpan data tentang karyawan perusahaan dan dimulai dengan membuat tabel karyawan berikut:

| employee_id | name  | job_code | job       | state_code | home_state |
|-------------|-------|----------|-----------|------------|------------|
| E001        | Alice | J01      | Chef      | 26         | Michigan   |
| E001        | Alice | J02      | Waiter    | 26         | Michigan   |
| E002        | Bob   | J02      | Waiter    | 56         | Wyoming    |
| E002        | Bob   | J03      | Bartender | 56         | Wyoming    |

**Struktur Tabel Awal (Belum Dinormalisasi)**

## Normalisasi Data

# Langkah 1: First Normal Form (1NF)

Untuk mencapai 1NF, kita perlu memastikan bahwa setiap kolom berisi nilai-nilai atomik (tidak ada grup yang diulang) dan bahwa setiap baris dapat diidentifikasi secara unik.

- Identifikasi primary key (misalnya, `employee_id`).
- Pisahkan atribut yang memiliki nilai ganda (jika ada).

Dalam kasus kita, data sudah bersifat atomik. Semua entri bersifat atomik dan ada primary key komposit (`id_karyawan`, `kode_pekerjaan`) sehingga tabel berada dalam bentuk normal pertama (1NF).

## Normalisasi Data

### Langkah 1: First Normal Form (1NF)

Tabel Order

| OrderID | CustomerID | Product    | Quantity | Price |
|---------|------------|------------|----------|-------|
| 1       | 1          | Laptop     | 2        | 1500  |
| 2       | 2          | Smartphone | 1        | 800   |
| 3       | 1          | Tablet     | 1        | 300   |
| 4       | 3          | Laptop     | 1        | 1500  |

Tabel Customer

| CustomerID | CustomerName | CustomerEmail     |
|------------|--------------|-------------------|
| 1          | John Doe     | john@example.com  |
| 2          | Jane Smith   | jane@example.com  |
| 3          | Sarah Connor | sarah@example.com |

## Normalisasi Data

# Langkah 2: Second Normal Form (2NF)

Kriteria 2NF mengharuskan bahwa:

- Dataset harus berada dalam 1NF.
- Semua atribut non-primer harus bergantung sepenuhnya pada primary key.
- **primary key**: Kombinasi dari `employee_id` dan `job_code`.

Tabel `employee_roles`

| employee_id | job_code |
|-------------|----------|
| E001        | J01      |
| E001        | J02      |
| E002        | J02      |
| E002        | J03      |
| E003        | J01      |

Tabel `employees`

| employee_id | name  | state_code | home_state |
|-------------|-------|------------|------------|
| E001        | Alice | 26         | Michigan   |
| E002        | Bob   | 56         | Wyoming    |
| E003        | Alice | 56         | Wyoming    |

Tabel `jobs`

| job_code | job       |
|----------|-----------|
| J01      | Chef      |
| J02      | Waiter    |
| J03      | Bartender |

## Normalisasi Data

### Langkah 3: Third Normal Form (3NF)

Pada proses sebelumnya, home\_state masih bergantung pada state\_code. Jadi, jika Anda mengetahui state\_code, maka Anda dapat menemukan nilai home\_state. Untuk melangkah lebih jauh, kita harus memisahkannya lagi ke tabel lain untuk menjadikannya 3NF.

**Tabel employee\_roles**

| employee_id | job_code |
|-------------|----------|
| E001        | J01      |
| E001        | J02      |
| E002        | J02      |
| E002        | J03      |
| E003        | J01      |

**Tabel employees**

| employee_id | name  | state_code |
|-------------|-------|------------|
| E001        | Alice | 26         |
| E002        | Bob   | 56         |
| E003        | Alice | 56         |

**Tabel jobs**

| job_code | job       |
|----------|-----------|
| J01      | Chef      |
| J02      | Waiter    |
| J03      | Bartender |

**Tabel states**

| state_code | home_state |
|------------|------------|
| 26         | Michigan   |
| 56         | Wyoming    |

## *Denormalisasi Data*

# Data Denormalization

Denormalisasi adalah proses menggabungkan tabel yang telah dinormalisasi kembali menjadi satu atau lebih tabel untuk meningkatkan kinerja query, meskipun bisa meningkatkan redundansi data. Proses ini biasanya dilakukan di database yang lebih besar, di mana pembacaan data lebih sering terjadi daripada penulisan.

Contohnya anda dapat mengembalikan data yang telah dinormalisasi pada proses sebelumnya menjadi data yang tergabung dalam 1 tabel (seperti data sebelum dilakukan normalisasi).

### Keuntungan Denormalisasi:

1. **Meningkatkan Performa Query** → Meminimalkan **JOIN** antar tabel.
2. **Sederhana untuk Akses Data** → Data sudah tersedia dalam satu tabel.
3. **Mengurangi Latency** → Akses data lebih cepat untuk kebutuhan laporan atau analisis.



# JOIN

## JOIN

# Apa itu JOIN?

Untuk menghubungkan tabel dalam query, kita menggunakan pernyataan JOIN...ON. Pernyataan JOIN menghubungkan satu tabel ke tabel lain dalam database selama query, menggunakan nilai yang cocok di kolom yang kita tentukan di kedua tabel.

```
SELECT *  
FROM table_a JOIN table_b  
ON table_a.key_column = table_b.foreign_key_column
```

```
-- Example  
SELECT * FROM teachers  
JOIN School ON teachers.school_id=School.school_id
```

## JOIN

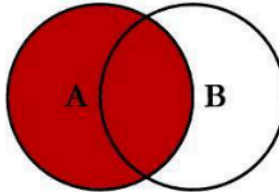
# Venn Diagram

Ada lebih dari satu cara untuk menggabungkan tabel dalam SQL.

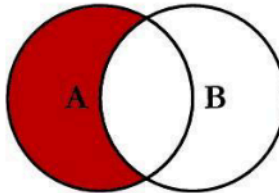
Jenis JOIN (penggabungan) yang akan Anda gunakan bergantung pada cara Anda ingin mengambil data.

Gambar di sebelah kiri menjelaskan berbagai jenis penggabungan.

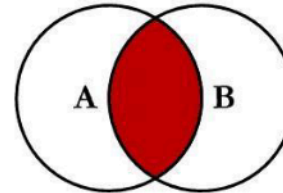
## SQL JOINS



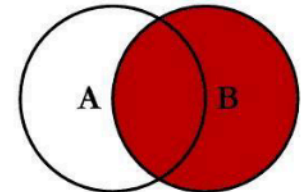
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
```



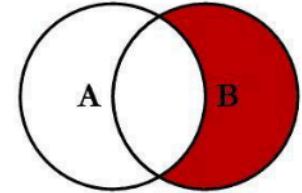
```
SELECT <select_list>
FROM TableA A
LEFT JOIN TableB B
ON A.Key = B.Key
WHERE B.Key IS NULL
```



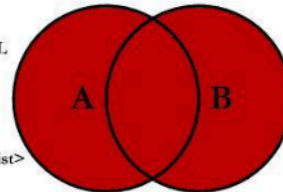
```
SELECT <select_list>
FROM TableA A
INNER JOIN TableB B
ON A.Key = B.Key
```



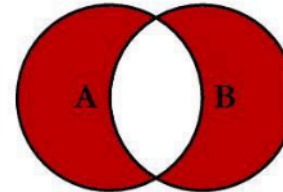
```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
RIGHT JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
```



```
SELECT <select_list>
FROM TableA A
FULL OUTER JOIN TableB B
ON A.Key = B.Key
WHERE A.Key IS NULL
OR B.Key IS NULL
```



# Subquery and CTE

## Subquery & CTE

# Apa itu Subquery?

Subquery adalah query yang bersarang di dalam pernyataan lain seperti SELECT, INSERT, UPDATE, atau DELETE.

Subquery juga dikenal sebagai inner query atau inner select, sedangkan pernyataan yang memuat subquery disebut outer select atau outer query.

**Subquery akan dieksekusi terlebih dahulu, kemudian hasilnya digunakan oleh query utama.**

Gambar disamping menunjukkan cara menggunakan subquery dalam klausa WHERE dari pernyataan SELECT untuk menemukan pesanan penjualan pelanggan yang berlokasi di New York.

```
SELECT
  order_id,
  order_date,
  customer_id
FROM
  sales.orders
WHERE
  customer_id IN (
    SELECT
      customer_id
    FROM
      sales.customers
    WHERE
      city = 'New York'
  )
ORDER BY
  order_date DESC;
```

```
Select * from orders o
join customers s on o.customer_id = s.customer_id
Where s.city = 'New York'
```

outer query

subquery

Sumber:  
<https://www.sqlservertutorial.net/sql-server-basics/sql-server-subquery/>

## Subquery & CTE

### ⚡ Non-Correlated

Efisiensi Tinggi: Running 1x

Subquery yang bersifat **independen** (berdiri sendiri). Komputer menghitung hasilnya sekali **saja** di awal, lalu digunakan oleh query utama.

```
SELECT nama, gaji FROM karyawan
WHERE gaji > (
  -- Subquery mandiri (hitung 1x)
  SELECT AVG(gaji) FROM karyawan
);
```

### 🔗 Correlated

Beban Tinggi: Perulangan (Loop)

Subquery yang **bergantung** pada data query utama. Harus dieksekusi berulang kali untuk setiap baris di query utama.

```
SELECT k1.nama, k1.gaji FROM karyawan k1
WHERE k1.gaji > (
  -- Subquery tergantung pada k1
  SELECT AVG(gaji) FROM karyawan k2
  WHERE k2.dept_id = k1.dept_id
);
```

## Subquery & CTE

# Apa itu CTE?

Common Table Expression (CTE) adalah cara lain untuk membuat query lebih modular dan mudah dibaca dengan mendefinisikan tabel sementara di awal query.

### Karakteristik CTE

- **Didefinisikan di Awal Query:** Menggunakan kata kunci **WITH**.
- **Dapat Digunakan Berulang Kali:** Membantu menghindari pengulangan logika.
- **Lebih Mudah Dibaca:** Terutama untuk query yang kompleks.

```
WITH NewYorkCustomers AS (  
  SELECT  
    customer_id  
  FROM  
    sales.customers  
  WHERE  
    city = 'New York'  
)  
SELECT  
  order_id,  
  order_date,  
  customer_id  
FROM  
  sales.orders  
WHERE  
  customer_id IN (  
    SELECT customer_id FROM NewYorkCustomers  
  )  
ORDER BY  
  order_date DESC;
```

```
-- 1. Kita buat "tabel sementara" (CTE) terlebih dahulu
WITH RataRataDepartemen AS (
    SELECT
        id_departemen,
        AVG(gaji) as rata_rata_gaji
    FROM karyawan
    GROUP BY id_departemen
)

-- 2. Query Utama (menggabungkan data karyawan dengan CTE)
SELECT
    k.nama,
    k.gaji,
    k.id_departemen,
    r.rata_rata_gaji
FROM karyawan k
JOIN RataRataDepartemen r
    ON k.id_departemen = r.id_departemen
WHERE k.gaji > r.rata_rata_gaji;
```



# Window Function

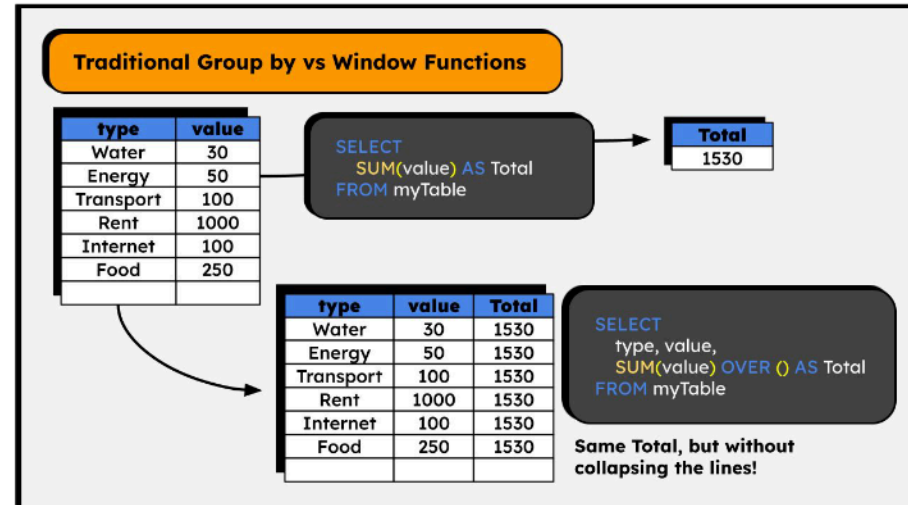
## Window Function

# Apa itu Window Function?

Window Function (fungsi jendela) dalam SQL adalah fungsi yang melakukan perhitungan pada sekelompok baris (sekumpulan data) yang terkait dengan baris saat ini, tanpa mengelompokkan data menjadi baris tunggal seperti **GROUP BY**.

Window Function memungkinkan kita:

- Melakukan perhitungan **agregat atau ranking** di atas sekumpulan baris (window).
- Tetap menampilkan semua baris **aslanya** di hasil akhir.



Sumber:

<https://medium.com/data-science/anatomy-of-windows-functions-08f04938b12b>

## Window Function

# Apa itu Window Function?

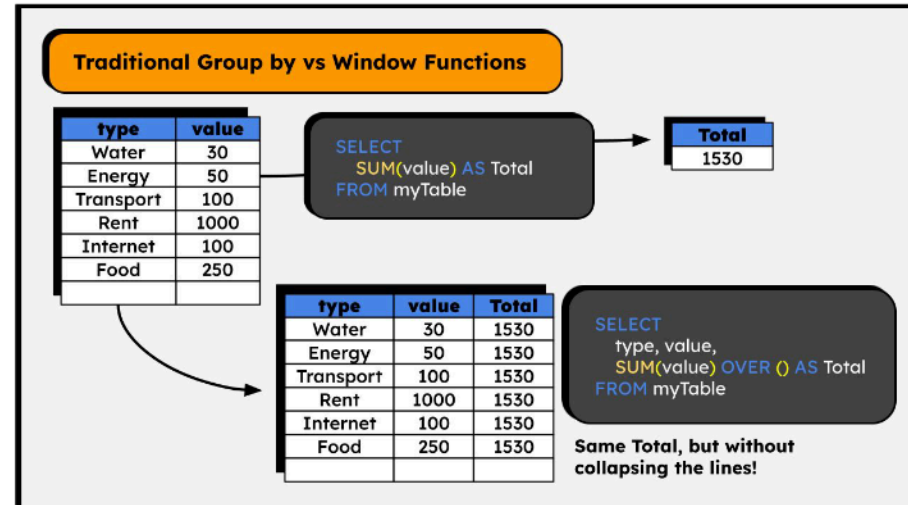
Klausula **OVER()** adalah syarat mutlak (wajib ada). Itulah "DNA" atau identitas utama dari sebuah Window Function. Tanpa kata **OVER()**, sebuah fungsi hanyalah fungsi biasa, bukan Window Function

Pada bagian ini kita akan fokus menggunakan fungsi:

**RANK**, **DENSE\_RANK** dan **ROW\_NUMBER**,

dengan klausula :

**ORDER BY** dan **PARTITION BY**.



## Window Function

# Latihan Windows Function

Misalnya kita punya tabel penjualan seperti berikut:

| sale_id | customer_name | city     | total_purchase |
|---------|---------------|----------|----------------|
| 1       | Andi Wijaya   | Jakarta  | 1500.00        |
| 2       | Budi Santoso  | Surabaya | 1800.00        |
| 3       | Citra Ayu     | Jakarta  | 1500.00        |
| 4       | Dewi Lestari  | Bandung  | 2100.00        |
| 5       | Eko Prasetyo  | Jakarta  | 1800.00        |
| 6       | Fajar Rahman  | Bandung  | 2100.00        |

```
CREATE TABLE sales (  
  sale_id INT PRIMARY KEY,  
  customer_name VARCHAR(100),  
  city VARCHAR(50),  
  total_purchase DECIMAL(10,2)  
);
```

```
INSERT INTO sales VALUES  
(1, 'Andi Wijaya', 'Jakarta', 1500),  
(2, 'Budi Santoso', 'Surabaya', 1800),  
(3, 'Citra Ayu', 'Jakarta', 1500),  
(4, 'Dewi Lestari', 'Bandung', 2100),  
(5, 'Eko Prasetyo', 'Jakarta', 1800),  
(6, 'Fajar Rahman', 'Bandung', 2100);
```

## Window Function

### Soal 1 - RANK()

Tampilkan daftar pelanggan, total pembelian, dan ranking berdasarkan total pembelian tertinggi menggunakan **RANK()**.

```
SELECT customer_name, total_purchase,  
       RANK() OVER (  
         ORDER BY total_purchase DESC  
       ) AS purchase_rank  
FROM sales;
```

| customer_name | total_purchase | purchase_rank |
|---------------|----------------|---------------|
| Dewi Lestari  | 2100.00        | 1             |
| Fajar Rahman  | 2100.00        | 1             |
| Budi Santoso  | 1800.00        | 3             |
| Eko Prasetyo  | 1800.00        | 3             |
| Andi Wijaya   | 1500.00        | 5             |
| Citra Ayu     | 1500.00        | 5             |

## Window Function

### Soal 2 - DENSE\_RANK()

Tampilkan daftar pelanggan, total pembelian, dan ranking **tanpa adanya loncatan peringkat** meskipun ada nilai sama (pakai **DENSE\_RANK()**).

```
SELECT customer_name, total_purchase,  
       DENSE_RANK() OVER (  
         ORDER BY total_purchase DESC) AS  
       purchase_dense_rank  
FROM sales;
```

| customer_name | total_purchase | purchase_dense_rank |
|---------------|----------------|---------------------|
| Dewi Lestari  | 2100.00        | 1                   |
| Fajar Rahman  | 2100.00        | 1                   |
| Budi Santoso  | 1800.00        | 2                   |
| Eko Prasetyo  | 1800.00        | 2                   |
| Andi Wijaya   | 1500.00        | 3                   |
| Citra Ayu     | 1500.00        | 3                   |

## Window Function

### Soal 3 - ROW\_NUMBER(), ORDER BY

Tampilkan daftar pelanggan, total pembelian, dan nomor urut unik untuk setiap baris berdasarkan total pembelian tertinggi (pakai **ROW\_NUMBER()**).

```
SELECT customer_name, total_purchase,  
       ROW_NUMBER() OVER (ORDER BY  
total_purchase DESC) AS row_num  
FROM sales;
```

| customer_name | total_purchase | row_num |
|---------------|----------------|---------|
| Dewi Lestari  | 2100.00        | 1       |
| Fajar Rahman  | 2100.00        | 2       |
| Budi Santoso  | 1800.00        | 3       |
| Eko Prasetyo  | 1800.00        | 4       |
| Andi Wijaya   | 1500.00        | 5       |
| Citra Ayu     | 1500.00        | 6       |

## Window Function

### Soal 4 - ROW\_NUMBER(), ORDER BY, PARTITION BY

Tampilkan daftar pelanggan, total pembelian, dan nomor urut unik untuk setiap tier berdasarkan total pembelian tertinggi (pakai **ROW\_NUMBER()** dan **PARTITION BY**).

```
SELECT customer_name, total_purchase,  
       ROW_NUMBER() OVER (PARTITION BY  
loyalty ORDER BY total_purchase DESC) AS  
row_num  
FROM sales;
```

| customer_name | loyalty  | total_purchase | row_num |
|---------------|----------|----------------|---------|
| Dewi Lestari  | Gold     | 2100.00        | 1       |
| Citra Ayu     | Gold     | 1500.00        | 2       |
| Fajar Rahman  | Silver   | 2100.00        | 1       |
| Budi Santoso  | Silver   | 1800.00        | 2       |
| Eko Prasetyo  | Silver   | 1800.00        | 3       |
| Andi Wijaya   | Platinum | 1500.00        | 1       |
| Citra Ayu     | Platinum | 1500.00        | 2       |



# Exercises

## Exercises

# Referensi Latihan

Gunakanlah notebook berikut ini untuk melakukan beberapa latihan sesuai dengan topik berikut:

1. Latihan JOIN, subquery dan CTE Dasar:  
<https://colab.research.google.com/drive/1NLM5IN72XNr8F2IkUozkuRCaUBpoT0Aj?usp=sharing>
2. Latihan Statistika Deskriptif dengan beberapa penggunaan Join:  
<https://colab.research.google.com/drive/1EUXgmPAAfZS2e2h9NOxF4W8H532WG2IA?usp=sharing>

# Terima Kasih



Hacktiv8id



Hacktiv8id



Hacktiv8



Hacktiv8 Indonesia